

SPRING WEBFLUX

En Spring WebFlux, Mono y Flux son los dos tipos de editores o publicadores reactivos principales provistos por Project Reactor para manejar flujos de datos asíncronos y no bloqueantes:

- **Mono:** Un publicador reactivo que emite a lo sumo un elemento (de 0 a 1 valor) o un error en el futuro. Es ideal para consultas únicas, como buscar un registro por su ID.
- **Flux:** Un publicador reactivo que emite una secuencia de múltiples elementos (de 0 a \$N\$ valores), un error o una señal de finalización. Es ideal para colecciones, flujos infinitos o streaming de datos en tiempo real.

¿Qué problema resuelven?

Resuelven el problema del bloqueo de hilos característico de los modelos tradicionales basados en un hilo por cada petición (como Spring MVC clásico).

En el desarrollo tradicional, cuando una aplicación realiza una operación costosa (como consultar una base de datos lenta o llamar a una API externa que tarda 5 segundos), el hilo del servidor se queda ocioso y bloqueado esperando la respuesta. Esto limita drásticamente la capacidad de concurrencia del sistema: si tienes 200 hilos disponibles y 200 peticiones esperando una consulta lenta, tu servidor colapsa y rechaza nuevas conexiones.

Con Mono, Flux y la programación reactiva:

- Se utiliza un modelo basado en eventos y un Event Loop con un número reducido de hilos fijos.
- Cuando una petición necesita esperar un dato, el hilo no se bloquea ni se queda esperando; en su lugar, el sistema registra una "receta" o suscripción de qué hacer cuando el dato llegue, y libera el hilo de inmediato para que pueda atender a miles de otras peticiones concurrentes.

¿Qué pasa si no se usan?

Si decides no usarlos y te mantienes en un enfoque tradicional (o si implementas WebFlux ignorando su naturaleza asíncrona):

- Menor escalabilidad y concurrencia: Tu aplicación consumirá muchos más recursos de memoria y CPU para mantener hilos bloqueados esperando respuestas de bases de datos o servicios externos.
- Cuellos de botella bajo alta carga: Ante un pico de tráfico, el servidor se quedará sin hilos disponibles rápidamente, provocando tiempos de espera elevados o fallos de conexión para los usuarios.
- Imposibilidad de hacer streaming real: Perderías la capacidad nativa de enviar datos en tiempo real fragmentados al cliente (como Server-Sent Events), viéndote obligado a esperar a que toda la información se acumule en memoria para entregarla de golpe como un bloque estático.

Spring WebFlux: Mono

En Spring WebFlux, un Mono es un contenedor reactivo que representa una promesa asíncrona de emitir a lo sumo un elemento (de 0 a 1 valor) o un error en el futuro, operando de manera completamente no bloqueante para optimizar los recursos del servidor.

```
20 @Repository
21 public class EmployeeRepository {
22
23     /** lo que tarda "la base de datos" en contestar. */
24     public static final Duration LATENCIA = Duration.ofMillis(5000); // 5 segundos
25
26     private final Map<Integer, Employee> tabla = new ConcurrentHashMap<>(Map.of(
27         1, new Employee(1, "Leslie", "Andrews", "leslie@luv2code.com"),
28         2, new Employee(2, "Emma", "Baumgarten", "emma@luv2code.com"),
29         3, new Employee(3, "Avani", "Gupta", "avani@luv2code.com"),
30         4, new Employee(4, "Yuri", "Petrov", "yuri@luv2code.com"),
31         5, new Employee(5, "Juan", "Ramirez", "juan@academyty.mx")
32     ));
33
34     /**
35      * Version reactiva: devuelve la RECETA de como conseguir el empleado.
36      *
37      * Ojo con lo que NO pasa aqui: al llamar a este metodo no se busca nada y no se
38      * espera nada. Se devuelve un Mono y el metodo termina de inmediato. La busqueda
39      * ocurre cuando alguien se suscribe -- y en un @RestController, quien se suscribe
40      * es Spring, no tu.
41      */
42     public Mono<Employee> findById(int id) {
43         return Mono.justOrEmpty(tabla.get(id)) // justOrEmpty: si no esta, Mono vacio
44             .delayElement(LATENCIA); // "la base de datos tarda"
45     }
46 }
```

La imagen muestra fragmentos de código Java en un entorno de desarrollo, específicamente una clase llamada `EmployeeRepository` anotada con `@Repository`.

En su contenido destacan los siguientes elementos:

Una constante de latencia configurada en 5 segundos (`Duration.ofMillis(5000)`) para simular el retraso de una base de datos.

Un mapa concurrente (`ConcurrentHashMap`) precargado con datos de prueba de cinco empleados (que incluyen nombres como Leslie Andrews, Emma Baumgarten, Avani Gupta, Yuri Petrov y Juan Ramírez).

Un método reactivo llamado `findById(int id)` que devuelve un objeto `Mono<Employee>`. Este método utiliza `Mono.justOrEmpty(tabla.get(id))` para obtener el registro de manera segura y aplica un `.delayElement(LATENCIA)` para simular la espera asíncrona explicada previamente.

```

16 @RestController
17 @RequestMapping("/api")
18 public class EmployeeRestController {
19
20     private static final Logger log = LoggerFactory.getLogger(EmployeeRestController.class);
21
22     private final EmployeeRepository repo;
23
24     public EmployeeRestController(EmployeeRepository repo) {
25         this.repo = repo;
26     }
27
28     /**
29      * Un empleado por id.
30      *
31      * Proyecto 15: public Employee findById(...) -> el hilo espera aqui
32      * Proyecto 01: public Mono<Employee> findById(...) -> devuelve el plan y suelta el hilo
33      */
34     @GetMapping("/employees/{id}")
35     public Mono<Employee> findById(@PathVariable int id) {
36         log.info("-> pediste el empleado {} (hilo: {})", id, Thread.currentThread().getName());
37
38         return repo.findById(id)
39             .doOnNext(e -> log.info("<- llego {} (hilo: {})",
40                 e.firstName(), Thread.currentThread().getName()))
41             .switchIfEmpty(Mono.error(new RuntimeException(
42                 HttpStatus.NOT_FOUND, "No existe el empleado " + id)));
43     }
44

```

La imagen muestra un controlador REST en Java utilizando Spring WebFlux, llamado EmployeeRestController y mapeado a la ruta /api.

Elementos clave del código:

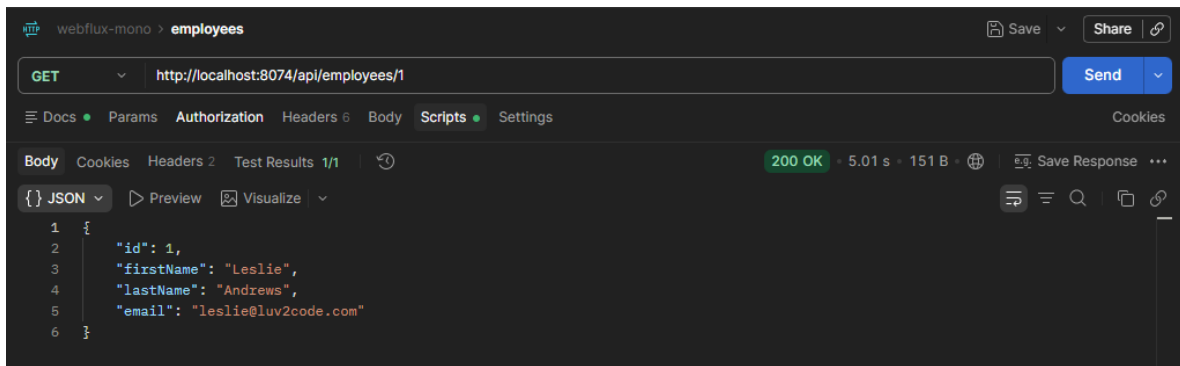
Inyección de dependencias: Cuenta con un constructor que inyecta una instancia de EmployeeRepository (repo).

Endpoint HTTP GET: El método findById(@PathVariable int id) está mapeado a /employees/{id} y devuelve un Mono<Employee>.

Trazabilidad (Logs):

- Imprime un registro (log.info) al recibir la petición indicando el ID solicitado y el hilo actual que atiende la solicitud.
- Utiliza .doOnNext(...) para registrar en los logs cuando el elemento llega con éxito, mostrando el nombre del empleado y el hilo encargado en ese momento.

Manejo de errores: Emplea .switchIfEmpty(...) para lanzar una excepción de tipo RuntimeException con un código HTTP 404 Not Found y el mensaje "No existe el empleado [id]" en caso de que el repositorio no encuentre el registro.



La imagen muestra una prueba de cliente HTTP realizando una petición a la API desarrollada previamente.

Método y URL: Se ejecutó una petición GET a la ruta `http://localhost:8074/api/employees/1`.

Código de Estado: La respuesta fue 200 OK, indicando que la operación fue exitosa.

Tiempo: Tomó 5.01 segundos (reflejando la latencia simulada en el repositorio).

Cuerpo de la Respuesta (JSON): Se muestra el objeto del empleado correspondiente al ID 1.

```
64  /**
65   * El canal de error. Este endpoint SIEMPRE falla, para que veas como se maneja
66   * un error sin un solo try/catch: el error viaja por el flujo, no por la pila.
67   */
68  @GetMapping("/employees/{id}/boom")
69  public Mono<Employee> boom(@PathVariable int id) {
70      return repo.findById(id)
71          .flatMap(e -> Mono.<Employee>error(new IllegalStateException("truena a proposito")))
72          .onErrorResume(ex -> {
73              log.warn("me lo comi: {}", ex.getMessage());
74              return Mono.just(new Employee(-1, "Plan", "B", "fallback@academyty.mx"));
75          });
76  }
77
```

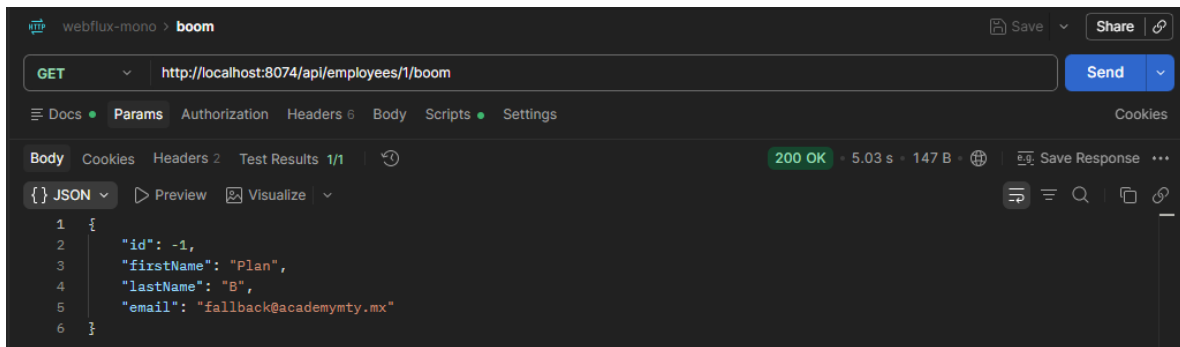
La imagen muestra un endpoint en un controlador de Spring WebFlux, diseñado específicamente para demostrar el manejo de errores en la programación reactiva.

Comentarios explicativos: Indica que el endpoint siempre falla a propósito para mostrar cómo se gestionan los errores a través del flujo de datos sin utilizar bloques try/catch tradicionales.

Mapeo de la ruta: Está anotado con `@GetMapping("/employees/{id}/boom")` y devuelve un `Mono<Employee>`.

Simulación de error controlado: Utiliza `.flatMap(...)` para forzar un error lanzando un `IllegalStateException`.

Manejo de recuperación (onErrorResume): Intercepta el error generado, imprime una advertencia en los registros (`log.warn`) con el mensaje del error y retorna un valor alternativo de respaldo (fallback) representado por un objeto `Employee` con datos predeterminados.



La imagen muestra una prueba de cliente HTTP ejecutando una petición al endpoint de manejo de errores que vimos anteriormente.

Detalles de la petición y respuesta:

Método y URL: Se realizó una petición GET a la ruta `http://localhost:8074/api/employees/1/boom`.

Código de Estado: La respuesta fue 200 OK. Aunque el endpoint forzó un error internamente, gracias al operador `.onErrorResume()` el sistema pudo recuperarse de forma controlada.

Cuerpo de la Respuesta (JSON): Se muestra exactamente el objeto de respaldo (fallback) configurado en el código cuando ocurre el error.

```

35     public record Hilo(String hilo, int hilosDisponibles, String pista) {}
36
37     @GetMapping("/api/hilo")
38     public Mono<Hilo> hilo() {
39         return Mono.just(new Hilo(
40             Thread.currentThread().getName(),
41             Runtime.getRuntime().availableProcessors(),
42             "Llama varias veces: se repiten los mismos nombres. Eso es el event loop."
43         ));
44     }
45

```

La imagen muestra un fragmento de código Java en Spring WebFlux diseñado para demostrar visualmente el funcionamiento del Event Loop e inspeccionar el hilo que procesa la solicitud HTTP.

Definición del Record (Hilo): Define una estructura inmutable (record) de Java llamada Hilo con tres campos.

Endpoint HTTP GET: Anotado con `@GetMapping("/api/hilo")`, el método `hilo()` devuelve un `Mono<Hilo>`.

Creación del resultado (Mono.just): Instancia el record Hilo con la siguiente información:

- `Thread.currentThread().getName()` para obtener el nombre exacto del hilo que está atendiendo la petición (por ejemplo, hilos tipo reactor-http-nio-X).
- `Runtime.getRuntime().availableProcessors()` para consultar el número de procesadores.
- El texto: "Llama varias veces: se repiten los mismos nombres. Eso es el event loop.", el cual sirve para evidenciar cómo WebFlux reutiliza un pequeño grupo fijo de hilos en lugar de crear un hilo nuevo por cada petición como ocurre en Spring MVC tradicional.

Spring WebFlux: Flux

En Spring WebFlux, un Flux es un publicador reactivo que representa una secuencia asíncrona capaz de emitir múltiples elementos (de 0 a ∞ valores), un error o una señal de finalización, permitiendo procesar flujos de datos grandes o colecciones de manera completamente no bloqueante.

```
18 @Service
19 public class SensorService {
20
21     private static final DateTimeFormatter HORA = DateTimeFormatter.ofPattern("HH:mm:ss");
22
23     /** Cada cuanto llega una lectura nueva. */
24     public static final Duration CADENCIA = Duration.ofSeconds(1);
25
26     /**
27      * El flujo crudo: una lectura por segundo, sin fin.
28      *
29      * La temperatura sigue una onda: sube y baja entre ~18 y ~34 grados en ciclos
30      * de 20 segundos. Es a propósito, no aleatorio: así las alertas se disparan de
31      * forma predecible y puedes comprobar que el filtro hace lo que dice.
32      */
33     public Flux<Lectura> lecturas() {
34         return Flux.interval(CADENCIA)
35             .map(this::medir);
36     }
37
38     private Lectura medir(long n) {
39         double celsius = 26 + 8 * Math.sin(2 * Math.PI * n / 20.0);
40         return new Lectura(
41             n,
42             "sensor-A",
43             Math.round(celsius * 10) / 10.0,
44             LocalTime.now().format(HORA));
45     }
46 }
```

La imagen muestra un fragmento de código Java correspondiente a una clase de servicio llamada `SensorService` anotada con `@Service`, diseñada para generar un flujo continuo de datos simulados usando un Flux.

Método público reactivo (lecturas):

- Devuelve un `Flux<Lectura>`.
- Utiliza `Flux.interval(CADENCIA)` para emitir valores periódicamente y los transforma mediante `.map(this::medir)`.

Método privado de medición (medir):

- Recibe un índice de tiempo `long n` y calcula una temperatura en grados Celsius utilizando una función trigonométrica (`Math.sin`).

- Retorna una nueva instancia del objeto Lectura con el identificador del incremento, el nombre del sensor ("sensor-A"), la temperatura redondeada a un decimal y la marca de tiempo actual formateada.

```

32●  /**
33   * (A) JSON normal.
34   *
35   * Spring se suscribe, JUNTA las 5 lecturas, y cuando el flujo termina manda
36   * el array de golpes. El navegador se queda en blanco 5 segundos y de repente
37   * aparece todo. Es indistinguible de un List<Lectura> del proyecto 15.
38   *
39   * curl http://localhost:8075/api/lecturas
40   */
41●  @GetMapping(produces = MediaType.APPLICATION_JSON_VALUE)
42  public Flux<Lectura> comoJson() {
43      log.info("[JSON] el cliente esperara a que terminen las 5 lecturas");
44      return sensor.lecturas().take(5);
45  }
46
47●  /**
48   * (B) El MISMO Flux, servido como stream de eventos.
49   *
50   * Ahora Spring NO junta nada: cada lectura sale por el cable en cuanto existe.
51   * El navegador pinta una linea por segundo. ESTA es la diferencia que un
52   * List<T> no puede darte.
53   *
54   * curl -N http://localhost:8075/api/lecturas/stream
55   *      ^^ la -N es imprescindible: desactiva el buffer de curl
56   */
57●  @GetMapping(path = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
58  public Flux<Lectura> comoStream() {
59      log.info("[STREAM] el cliente recibira una lectura por segundo");
60      return sensor.lecturas().take(20);
61  }
62

```

La imagen muestra dos endpoints diferentes en un controlador de Spring WebFlux que utilizan el mismo Flux<Lectura> del servicio de sensores, pero con formas distintas de entregar la respuesta al cliente:

1. Endpoint JSON normal (comoJson)

Mapeo: @GetMapping(produces = MediaType.APPLICATION_JSON_VALUE) (por defecto en la raíz).

Funcionamiento: Spring se suscribe al flujo, acumula las 5 lecturas especificadas con .take(5) y envía el arreglo completo de golpe.

Comportamiento: El cliente (como el navegador) se queda esperando en blanco durante los 5 segundos que dura la acumulación y luego recibe todo de una sola vez. Se comporta de manera idéntica a una lista tradicional (List<T>).

2. Endpoint Stream de Eventos (comoStream)

Mapeo: @GetMapping(path = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE) (Server-Sent Events / SSE).

Funcionamiento: Spring no acumula nada; cada lectura sale por el cable exactamente en cuanto se genera (en este caso tomando hasta 20 elementos con `.take(20)`).

Comportamiento: El cliente recibe e imprime una línea por segundo en tiempo real, demostrando la capacidad de streaming bidireccional/continuo que caracteriza a la programación reactiva y que una lista tradicional no puede ofrecer.

```
67  /**
68   * filter() sobre un flujo vivo: solo las lecturas por encima del umbral.
69   *
70   * Como la temperatura es una onda de 20 segundos, veras rachas de alertas
71   * y luego silencio. El flujo NO se para en los silencios: sigue corriendo,
72   * simplemente no emite.
73   *
74   * curl -N "http://localhost:8075/api/lecturas/alertas?umbral=30"
75   */
76  @GetMapping(path = "/alertas", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
77  public Flux<Lectura> alertas(@RequestParam(defaultValue = "30") double umbral) {
78      return sensor.lecturas()
79          .filter(l -> l.celsius() > umbral)
80          .doOnNext(l -> Log.warn("ALERTA {} C", l.celsius()));
81  }
82
83  /**
84   * takeUntil(): el ejemplo literal del mapa mental,
85   * "vigila el precio de la accion hasta que baja de 100".
86   *
87   * Aqui: manda lecturas hasta que la temperatura baja del umbral, y ahí
88   * emite onComplete y cierra la conexión sola. Miralo con -N: el curl
89   * TERMINA solo, sin Ctrl-C.
90   *
91   * curl -N http://localhost:8075/api/lecturas/hasta/20
92   */
93  @GetMapping(path = "/hasta/{umbral}", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
94  public Flux<Lectura> hasta(@PathVariable double umbral) {
95      return sensor.lecturas()
96          .takeUntil(l -> l.celsius() < umbral)
97          .doOnComplete(() -> Log.info("bajo de {} C: onComplete y cierro", umbral));
98  }
```

La imagen muestra dos endpoints avanzados en Spring WebFlux que aplican operadores de filtrado y control de flujo sobre el Flux de lecturas continuas:

1. Endpoint de Alertas filtradas por umbral (alertas)

Mapeo: `@GetMapping(path = "/alertas", produces = MediaType.TEXT_EVENT_STREAM_VALUE)` con un parámetro opcional `umbral` (por defecto 30).

Funcionamiento: Utiliza el operador `.filter(l -> l.celsius() > umbral)` para evaluar el flujo en tiempo real y emitir únicamente aquellas lecturas cuya temperatura supere el límite establecido.

Comportamiento: Como la temperatura sigue una onda sinusoidal, produce rachas de alertas alternadas con silencios (donde el flujo sigue corriendo pero simplemente no emite valores hacia el cliente), registrando una advertencia en los logs mediante `.doOnNext(...)` cada vez que se dispara una alerta.

2. Endpoint de límite condicional (hasta)

Mapeo: `@GetMapping(path = "/hasta/{umbral}", produces = MediaType.TEXT_EVENT_STREAM_VALUE)` recibiendo el umbral como variable de ruta.

Funcionamiento: Aplica el operador `.takeUntil(l -> l.celsius() < umbral)`, el cual transmite las lecturas de manera continua hasta que se cumple la condición de que la temperatura baje del valor indicado.

Comportamiento: En ese momento exacto, el flujo emite una señal de finalización (`onComplete`), cierra la conexión automáticamente por sí solo (sin necesidad de interrumpir manualmente con Ctrl+C) y ejecuta un mensaje informativo en los logs a través de `.doOnComplete(...)`.